Low-Level Programming
CSD3116
Trimester 1, AY2026/2027
Week 1
Team Lab Assignment
Real-Time, Interactive, Distributed and Secure Simulation

Dr Khoo Teck Ping
Assistant Professor
DigiPen Institute of Technology Singapore
31 August 2026

Outline

1. **Objectives**
2. Background
3. Baseline programs
4. Tasks
5. Deliverables
6. Grading

Objectives

1. Learning through peer support, through a team assignment
2. Apply concepts covered in class
    1. Code optimization
    2. Threading
    3. Networking
    4. Security
3. Learn Data Distribution Service (DDS)
4. Learn beyond this course, eg game mechanics, game AI
5. Build a tech portfolio, to prepare for IWSP and future employment

Outline

1. Objectives
2. **Background**
3. Baseline programs
4. Tasks
5. Deliverables
6. Grading

Data Distribution Service (DDS) [1]

1. DDS is a real-time, machine-machine, networking middleware standard from Object Management Group (OMG) [2]
2. Goals
    1. High-performance
        1. Typically uses UDP instead of TCP
        2. Low packet overhead and no re-transmission increases throughput
        3. Reliability provided at DDS layer
    2. Inter-operable
    3. Real-time
    4. Scalable
    5. Publish and Subscribe
3. Widely used in aerospace & defense [3], autonomous vehicles, industrial automation, healthcare, energy and power systems, and ROS2 (Robot OS 2) [4]

1. https://en.wikipedia.org/wiki/Data_Distribution_Service
2. https://en.wikipedia.org/wiki/Object_Management_Group
3. https://www.omgwiki.org/ddsf/doku.php?id=ddsf:public:applications:simulation_and_test
4. https://github.com/ros2

Outline

1. Objectives
2. Background
3. **Baseline programs**
4. Tasks
5. Deliverables
6. Grading

Baseline programs

1. A *server.c* and *client.c* files are provided
    1. They communicate using DDS
    2. A *messages.idl* (interface description language) [1] file is provided to define the DDS messages
2. Start the server first
3. Start the clients one by one (max five clients)
    1. An 800x600 graphics window shall appear
    2. The server will assign the first client as Player 1, the second as Player 2, etc
    3. A 50x50 labelled player square will appear at a random position in the window
        1. It will not intersect with the window's edges or another player's square
4. When a player clicks inside the client window, the player's square can be moved up, down, left and right using the arrow keys
    1. The player's square will not intersect with the window edges or another player's square
    2. All players will see all squares updated in real-time

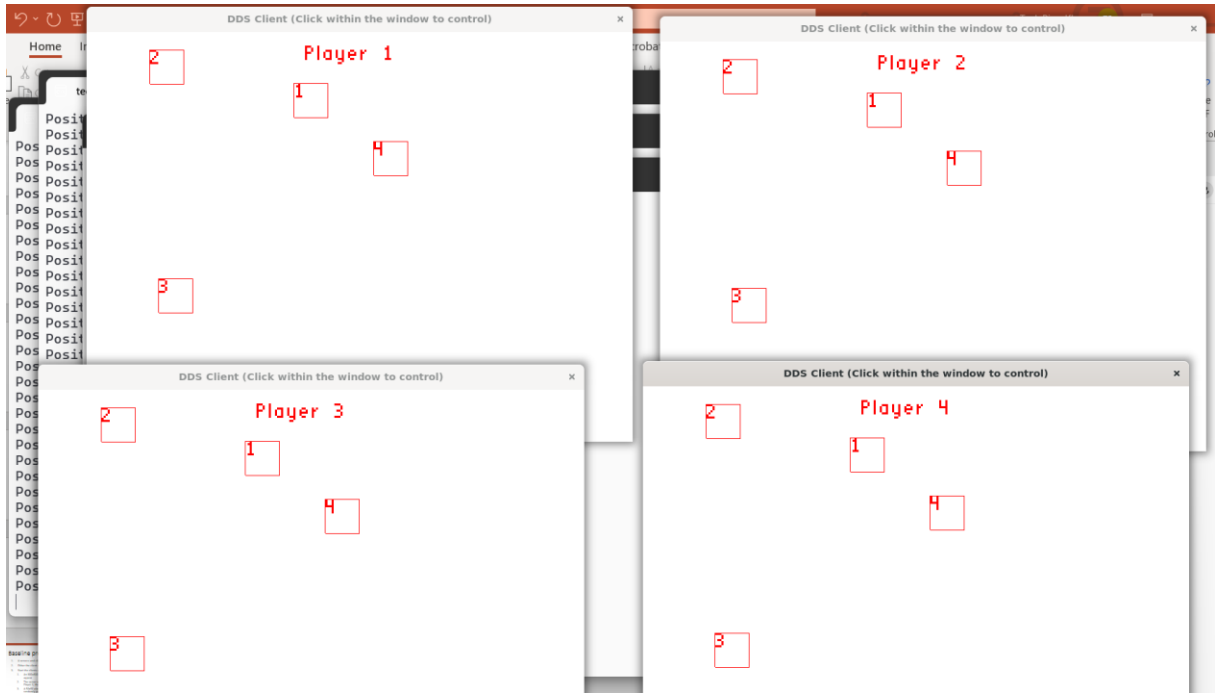


Figure 1: Example run with four clients

1. https://en.wikipedia.org/wiki/Interface_description_language

Setting up

1. Open WSL
2. Install OpenGL development files
    1. *sudo apt install libgl1-mesa-dev*
3. Graphics Library Framework (GLFW) [1] shall be used
4. Install GLFW headers and libraries
    1. *sudo apt install libglfw3-dev*
5. Cyclone DDS [2] shall be used – install using:
    1. *sudo apt update*
    2. *sudo apt install -y cyclonedds-dev cyclonedds-tools*

1. https://www.glfw.org/
2. https://github.com/eclipse-cyclonedds/cyclonedds

Building and Running

1. To create the DDS files *messages.h* and *messages.c*
    1. *idlc –l c messages.idl*
2. To build and run the server
    1. *gcc -o server server.c messages.c $(pkg-config --cflags --libs CycloneDDS)*
    2. *./server*
3. To build and run the client
    1. *gcc client.c messages.c -o client $(pkg-config --cflags --libs glfw3) -lGL $(pkg-config --cflags --libs CycloneDDS) -pthread –lm*
    2. *./client*

```
1  //idlc -l c messages.idl
2
3  struct JoinRequest
4  {
5      boolean b_dummy;
6  };
7
8  struct JoinResponse
9  {
10     //IDL needs int width to be clearly defined
11     long int_player_id;
12 };
13
14 struct Input
15 {
16     long int_player_id;
17     char ch_direction;
18 };
19
20 struct Position
21 {
22     long int_player_id;
23     long int_x;
24     long int_y;
25 };
```

Figure 2: messages.idl

1. https://www.glfw.org/
2. https://github.com/eclipse-cyclonedds/cyclonedds

Game State

1. The game state (ground truth) is only updated by the server, using an array as shown
2. The clients maintain the same array by getting position messages from the server
    1. Server will only send position messages to active clients
    2. Clients will only render the squares of active players
3. **The player ID is the array index + 1**
    1. Enables direct lookup and update
4. Client shutdown and leaving the game are not supported

Index	Active	X	Y
Integer	Boolean	Integer	Integer

Figure 3: Array for the Game State

Publishing and Subscribing

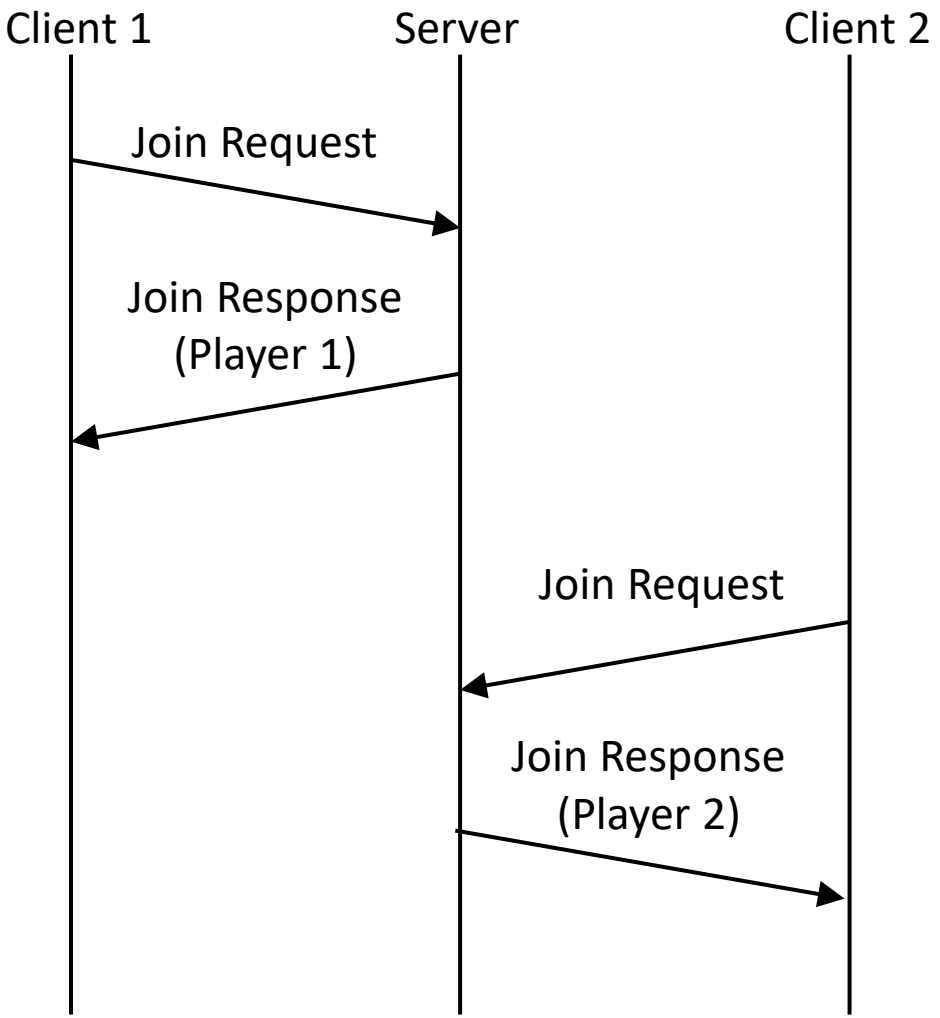


Figure 4a: Join Request and Response

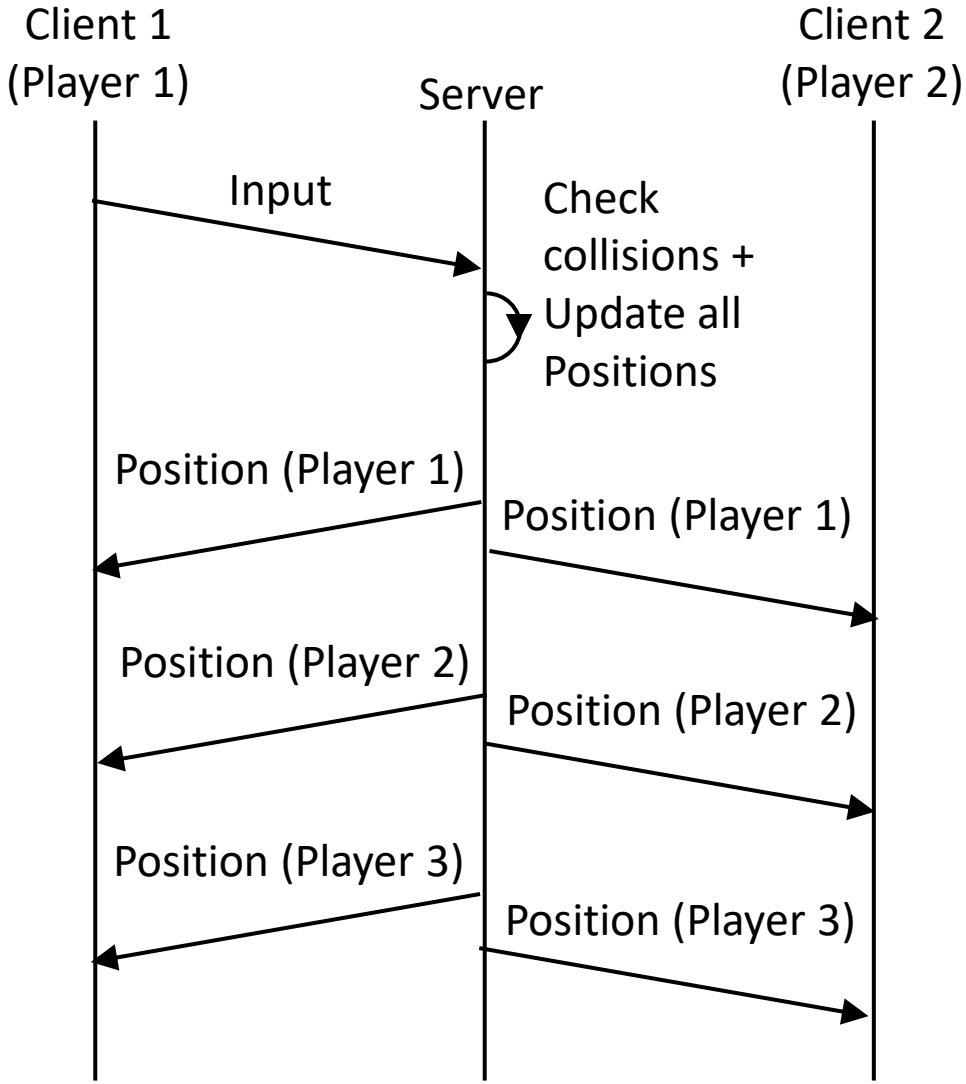


Figure 4b: Client input and server position

Threading

1. Server and client each have a main thread and a DDS worker thread
    1. **A single thread will cause delays in server state processing or client window input/rendering, eg when DDS traffic is delayed**
    2. Server main thread is game state maintenance
    3. Client main thread is OpenGL rendering and GLFW input handling
2. A thread uses a mutual exclusion (mutex) lock shared variables for read/write, and unlock it afterwards
    1. Prevents the other thread from reading it while it is being changed, or changing it while it is being read
    2. The mutex lock and unlock must match

```
// Check for valid join request
if ((rc > 0) && (infos_join[0].valid_data))
{
    printf("Join request received\n");
    // Signal the main thread to check if
    // number of active players reached
    pthread_mutex_lock(&mutex);
    b_player_join = true;
    pthread_mutex_unlock(&mutex);
}
```

Figure 5a: Example use of mutex in *server.c*

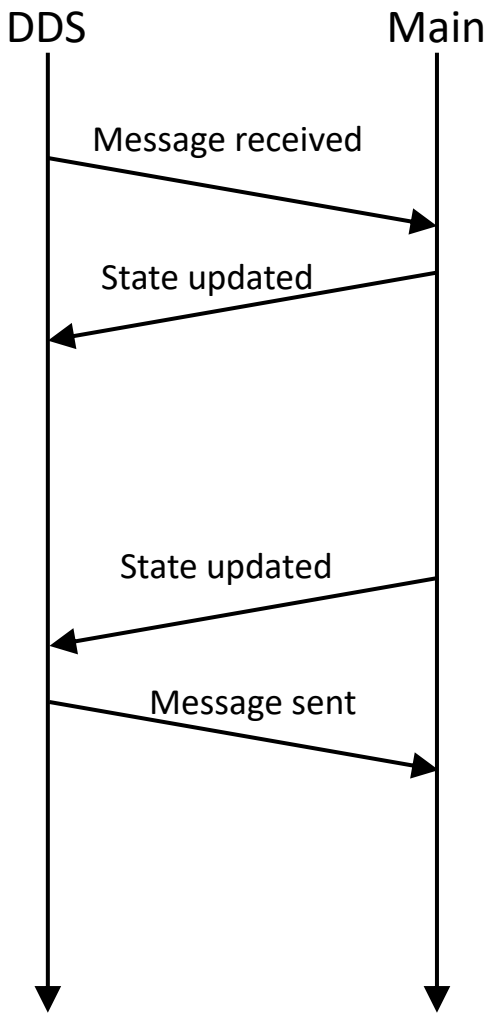


Figure 5b: Server and Client each have a main and a DDS thread

Outline

1. Objectives
2. Background
3. Baseline programs
4. **Tasks**
5. Deliverables
6. Grading

Tasks

1. Form a team of minimum two, maximum three, and appoint a team leader
    1. **By Monday, 14 September 2026**
    2. After the deadline, a meeting shall be arranged for students without a team to form teams
2. Design and build a game from the baseline programs
3. Game requirements:
    1. Must be 2D/3D animation-based
    2. Must use WSL, OpenGL, GLFW and Cyclone DDS
        1. macOS is Unix-like and can be used in place of WSL
    3. Must support minimum two players
    4. Must be real-time, interactive, distributed and secure
    5. Should improve on the baseline programs, eg support client leaving the game
    6. Concepts covered in class must be applied, eg
        1. How to write optimized code, eg improve threading, avoid repeated computation in a loop
        2. How to implement security
            1. Authentication of clients
            2. Encryption of DDS traffic

Outline

1. Objectives
2. Background
3. Baseline programs
4. Tasks
5. **Deliverables**
6. Grading

Deliverables

1. **Deadline: Week 10 - Friday, 6 November 2026**
2. **Submission: In Moodle**
3. **Assignment is AI-integrated**
    1. **Use of AI, such as a summary of the prompts used, how the code generated was tested, must be declared in any deliverable**
4. Source code (.zip)
    1. Must be clearly commented
5. A set of presentation slides (.pdf) describing:
    1. Instructions to build and execute the program
    2. Which team member did what
    3. The game concept, mechanics and design, eg software engineering diagrams
    4. The code optimization done, validation using tools such as profilers is recommended
    5. The security features implemented
    6. **If AI was used to generate code, you must prove that the code was tested, eg:**
        1. **Walkthrough of the code**
        2. **A defense of accepted output**
6. A video (.mp4, max 200MB):
    1. Maximum five minutes
    2. Has to be narrated by all team members (no use of AI-audio)
    3. Video of team members narration is recommended (refer to video grading rubrics)
    4. Can be AI-captioned
    5. Identify your team and yourselves
    6. Demonstrate your game
    7. Describe the code optimization done and the security features implemented

Outline

1. Objectives
2. Background
3. Baseline programs
4. Tasks
5. Deliverables
6. **Grading**

Grading

Deliverable	Weighting/%	Remarks
Program	40	Can be executed = 20% Overall game look-and-feel = 10% Implementation of optimization and security = 40% Code structure and modularity = 10% Readability = 10% Comments = 10%
Presentation slides	20	Refer to rubrics
Video	20	Refer to rubrics
Peer Evaluation	20	Instructions to be communicated nearer to the deadline
Total	100	

Presentation Slides

	Clarity = 25%	Structure = 25%	Technical Content = 50%
>=90	Exceptionally clear slides. Simple design, large enough font, and not too much material on slides. A professional quality presentation.	Structure of the presentation makes understanding of the technical arguments exceptionally clear.	Excellent quality and quantity of high-level technical content. Analysis demonstrates critical understanding of the project.
>=70	Clear slides with perhaps occasional flaws (small font size, inappropriate colour schemes, etc), but overall impressive presentation.	Very well-structured presentation with everything where it should be to provide clarity.	Very good level of technical content and analysis to give the audience a clear account of the project.
>=60	Errors on some slides, but overall still clear. Flaws do not detract significantly from content.	Well-structured presentation, but perhaps one or two slides are misplaced.	Good level of technical content and analysis, with only a small amount of superfluous information.
>=40	Errors on several slides, but not of a significant nature. A reasonable effort, but flaws have detracted from presentation.	Some presentation elements are unclear as the structure is slightly confused.	Satisfactory level of technical content and analysis, with some irrelevant material or too much general background information.
<40	Significantly flawed slides with basic errors (small font size, too much material on slides, over-elaborated design, inappropriate colour schemes, etc).	Badly structured presentation giving a confused picture of the project, making it difficult to follow arguments.	Mostly irrelevant technical content, with limited or no analysis.

Video

Mark	Standard	Clarity & Organization 25%	Technical Content 25%	Demonstration & Visuals 25%	Communication Skills 25%
>=90	Excellent	Video is well-structured, logically organized, and easy to follow. Transitions are smooth and content flows naturally.	Engineering concepts are explained accurately, in-depth, and supported with relevant examples.	High-quality visuals, diagrams, or demonstrations enhance understanding and are well-integrated.	Clear, confident, and professional delivery with good pacing and audience engagement.
>=70	Good	Mostly clear with minor gaps in organization or transitions.	Mostly accurate, but minor errors or missing details.	Visuals support content well but could be more polished or better integrated.	Mostly clear, with minor pacing or clarity issues.
>=60	Satisfactory	Somewhat clear but lacks logical flow in parts. Transitions are rough.	Some technical inaccuracies or lack of depth in explanations.	Some visuals, but they are unclear, inconsistent, or not well-connected to content.	Understandable but lacks confidence, clarity, or enthusiasm.
>=40	Needs Improvement	Poorly structured, difficult to follow, and lacks logical progression.	Major inaccuracies, minimal technical depth, or missing key details.	Little to no visuals, or visuals do not effectively support the explanation.	Poor delivery, difficult to understand, lacks engagement.
<40	Unacceptable	No clear structure; video is confusing and disorganized.	Technical content is mostly incorrect, unclear, or missing.	No relevant visuals or demonstrations included.	Incoherent or monotonous, making it hard to follow.

End